

Live Deployment with CI

A standalone, deep-dive reference for Build 5 — every phase, every real bug the process itself surfaced, the mid-build data-anonymization pivot, and every verification result, in full. Complements (does not replace) the living cheat sheet and the two per-phase flow diagrams.

STATUS

Complete — all 3 phases, CI green end-to-end

COMMITTS

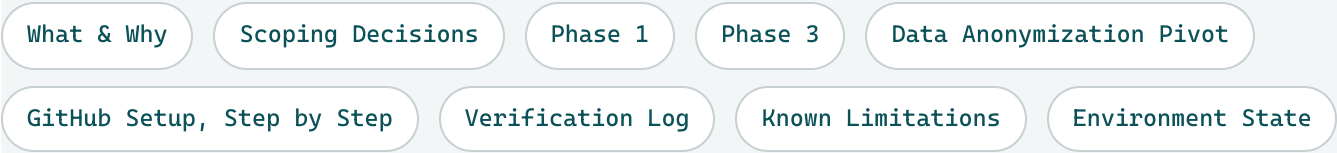
08b0437 → cbc307

CONTINUES

Build 4 (agent observability & eval harness)

PRECEDES

Build 6 (optional — Streamlit front end)



What Was Built, and Why

Every prior build deepened the agent itself. Build 5 makes the project reproducible and verifiable by something other than this one machine: a public GitHub repository with a CI workflow that proves — on every push, from a genuinely empty environment — that the schema builds, the data pipeline runs, the test suite passes, and (on a relevant merge to `main`) the agent's own eval harness still passes too.

The most consequential work this build wasn't in the original scope at all: a mid-build realization that the small CI-only data fixture from Phase 1 didn't actually satisfy the real anonymization requirement, leading to a full replacement of the real/fixture split with one official synthesized dataset, and three separate `git filter-repo` history rewrites to make sure nothing that shouldn't be public ever was. That work is documented in full in its own section below, not folded quietly into the phase write-ups, because it's the part most worth learning from.

Trust boundary, unchanged from Build 1, now proven on a machine that isn't this one. Every phase of this build exists to answer one question with evidence instead of assumption: does this actually work from zero, or does it only work here, on this laptop, because of six months of accumulated state nobody wrote down? Three separate real bugs (Phase 1) and a data-exposure gap (the anonymization pivot) all trace back to exactly that question.

Scoping Decisions, Made Before Any Code Was Written

Two real forks were flagged at the start of this build and resolved deliberately, not defaulted into.

1. **"Live deployment" means CI-only, not a public-facing app.** GitHub Actions validates migrations + the full test suite + a gated eval run; "deployment" means reproducibly buildable via Docker Compose, no public URL. The actual public-facing demo is Build 6's job (a Streamlit front end), not this build's — building both at once would be scope creep.
2. **The eval harness runs in CI, but gated — path-filtered and merge-to-main only, not every push.** The real per-run cost turned out to be cents (a handful of cases on `claude-haiku-4-5`, 2–4 turns typically) — so the actual tradeoff wasn't API dollars, it was `ANTHROPIC_API_KEY` becoming a live GitHub Actions secret plus noise on every WIP commit, versus catching a real regression before it merges. `pytest` runs on every push regardless, since it costs nothing.

Also decided: the new GitHub remote would be public. Matches the project's stated portfolio/placement goal — and the data had already been domain-masked specifically for this eventuality back in Build 2. (That masking turned out not to be sufficient on its own — see the anonymization section.)

1 GitHub Remote & CI Skeleton (Phases 1 + 2 combined in practice)

Goal: a public repo, a CI workflow proving the schema and full pipeline build from zero, and the existing pytest suite wired in — Phase 2 turned out to be a natural side effect of building Phase 1 properly, not a separate step.

```
.github/workflows/ci.yml - the core test job

services:
  postgres:
    image: pgvector/pgvector:pg17 # not plain postgres - schema needs the vector extension
    env:
      POSTGRES_USER: devuser
      POSTGRES_PASSWORD: ci_admin_pw # throwaway, scoped to this ephemeral container only
      POSTGRES_DB: appdb

steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-python@v5
  - run: pip install -r requirements.txt
  - # apply 001 through 011 in order, then ingest -> transform -> embed -> pytest
```

Bug 1 — no migration ever ran `CREATE EXTENSION vector`. Even on the `pgvector/pgvector:pg17` image, the extension had been enabled by hand, once, months ago, directly in a `psql` session when Build 3 was first set up — never captured in any committed file. `008_add_summary_embedding.sql`'s own comment even referred to "the vector extension (enabled Phase 1, step 1)" — a manual step that was never actually SQL. Every migration re-run against `pg_local` since then silently found the extension already present. It surfaced only the moment a genuinely fresh volume tried to apply these migrations from scratch — first a disposable local throwaway container, then GitHub Actions' real ephemeral runner.

Fixed: `CREATE EXTENSION IF NOT EXISTS vector`; added directly to `008_add_summary_embedding.sql`, idempotent so it's a no-op on a volume where it's already enabled.

Bug 2 — a pytest test silently depended on leftover local-only state.

`test_scratch_table_cap_blocks_new_tables_but_not_existing_ones` reused `agent_scratch.domain_avg_duration`, a table that exists only because of real hands-on Build 2.5 testing months earlier, never created by any script. On a fresh database it doesn't exist, so the test's "existing table" branch never actually exercised what it claimed to — `save_dataframe` correctly saw "table doesn't exist" plus the test's own monkeypatched `MAX_SCRATCH_TABLES = 0`, and correctly refused to create it, raising an unhandled `RuntimeError` inside a call the test expected to succeed.

Fixed: the test now creates its own throwaway table first (under a temporarily-raised `MAX_SCRATCH_TABLES` via monkeypatch), making it fully self-contained regardless of what else exists in `agent_scratch`.

Bug 3 — `api_test.py`, a stray non-test script, was silently collected and executed by bare pytest. This was the very first Anthropic API call ever made in this project, predating `agent.py` — a scratch connectivity check with no test functions, just a module-level `client.messages.create(...)`. It matched pytest's `*_test.py` discovery glob even though every real test file uses only `test_*.py`. Locally, a valid `ANTHROPIC_API_KEY` in `.env` meant this call had silently succeeded (and quietly spent real API credit) on every bare pytest invocation, ever — nothing to fail, so nothing for pytest to ever flag as wrong. It surfaced the moment pytest ran somewhere without that key already present: the first real Actions run, correctly, since the eval harness's secret wasn't configured until Phase 3.

Fixed: `api_test.py` deleted outright (fully superseded by `agent.py`'s real functionality, confirmed nothing else referenced it), plus a new `pytest.ini` (`python_files = test_*.py`) scoping discovery to the convention every real test file already follows, so no future stray script can repeat this.

Common thread across all three bugs: each was invisible specifically because `pg_local1`'s long-lived Docker volume and local `.env` had silently satisfied an undocumented or accidental prerequisite for a long time. Each surfaced only by deliberately testing against an environment with none of that accumulated state — first a disposable throwaway container, then GitHub Actions' actual runner. This is the concrete, project-specific argument for why CI closes a real gap rather than just replicating what local testing already covers.

3 Gated Eval Harness in CI

Goal: run the existing eval harness in CI too, but only on a relevant merge to main, per the scoping decision above.

```
.github/workflows/ci.yml - detect-changes job

- uses: actions/checkout@v4
  with:
    fetch-depth: 0 # paths-filter needs real git history to diff a push event

- uses: dorny/paths-filter@7b450fff21473bca461d4b92ce414b9d0420d706 # v4.0.2, pinned by SHA not tag
  id: filter
  with:
    base: ${ github.ref }
    filters: |
      eval:
        - 'agent.py'
        - 'tools.py'
        - 'eval_cases*.py'
```

A real design detail confirmed by research before writing this, not assumed: paths-filter can work purely via the GitHub API (no checkout needed) for pull_request events, but a plain push event needs real git history to compute a merge-base against the previous commit on the same branch — hence `fetch-depth: 0` and an explicit `base: ${ github.ref }`, not the default shallow clone.

```
.github/workflows/ci.yml - the gated eval job

eval:
  needs: [test, detect-changes] # never fires against an already-broken build
  if: github.ref == 'refs/heads/main' && needs.detect-changes.outputs.eval == 'true'
  # ...same migrate/ingest/transform/embed setup as the test job, then:
  - run: python run_eval.py
  - run: python run_eval.py eval_cases_round2
```

Real gap found while wiring this up: run_eval.py never set a non-zero exit code. It printed a full pass/fail report but always exited 0 — meaning this job could never actually go red on a genuine eval regression, only on an outright crash. A meaningless gate.

Fixed: `sys.exit(1)` if any case fails. Confirmed the fix actually works by watching a real CI run go red on a genuine failure (see the anonymization section — the same run that first surfaced this also proved the exit code

fires correctly).

Live-verified both halves of the gate for real, not assumed from reading the YAML: a comment-only touch to `eval_cases.py` correctly triggered the `eval` job on a real merge to `main`; a separate, unrelated push (touching only `ci.yml/run_eval.py`) correctly left it skipped, shown as - in the Actions UI.

The Data Anonymization Pivot

Not part of the original Build 5 scope — surfaced by a direct question mid-build ("should I just mask the whole file"), which led to discovering the existing approach didn't actually meet the real bar. Documented in full because it's the most consequential work in this build.

The Phase 1 CI fixture (30 rows, masked ID + domain only) didn't satisfy the real requirement. A closer look at what "anonymization" actually needed to protect against revealed the bar was broader than institution identity — **no real operational/proprietary data public**, not just "don't reveal who this is." Under that standard, the already-published fixture failed: real department codes, real dates/durations, and real equipment descriptions (e.g. CANON T3i, GVM 800D RGB LIGHT, only the embedded tag numbers scrubbed) were all still real, for those 30 rows, already public.

Resolution: one official synthesized dataset (data/ALLOCATION-synthesized.csv) replaces the real-file/fixture split entirely — used identically by local dev, CI, and the eval harness, so there's no separate fixture to keep in sync.

FIELD	TREATMENT
renewal_count, all 3 dates, duration, resource_count	kept real, unchanged, row-for-row – nothing proprietary in these alone
allocation_id	random unique 7-digit IDs (random.sample, not sequential – changed after explicit correction) – categorically different length from real 6-digit IDs
patron_department	1:1 fictional remap (DEPARTMENT_MAP) – exact distribution/plurality shape preserved; blanks (787 of 1535 rows) left as-is
patron_email domain	artschool.edu family → fixture.edu family, subdomain structure kept; gmail.com left as-is per explicit instruction (generic provider, not identifying)
summary	fully synthesized from a ~50-item pool spanning the same real equipment categories, different specific items; resource_count real items generated per row

The real department-code mapping is deliberately not in this public repo. Publishing real code → fake code side by side would let anyone reverse the masking. It lives in two git-ignored, local-only files: `DEPARTMENT_MAPPING.local.md` (human-readable) and `department_mapping.local.py` (the actual source `synthesize_dataset.py` imports at runtime, via `importlib.util` since a literal dot in a filename isn't a valid dotted import path). Same treatment this project already gives `CREDENTIALS.local.md`.

A real mistake, caught before it ever left the machine. The first version of `synthesize_dataset.py` hardcoded `DEPARTMENT_MAP` with the real codes as dictionary keys directly in the committed script — exactly the mapping meant to stay private. Caught by the same -S pickaxe verification used to confirm the credential purge, before this commit was ever pushed. Fixed via `git commit --amend` (safe here specifically because nothing had been shared yet), plus `git reflog expire --expire=now --all && git gc --prune=now` to fully prune the flawed blob locally.

A genuinely good side effect: the unified dataset resolved most of `eval_cases_round2.py`'s CI-portability problem for free. Of 3 originally-failing cases: the department-plurality case just needed its expected keyword updated to match the new synthetic department code (DRAW, same real count of 128, since the remap is 1:1 — the real code it replaced isn't reproduced here, same reasoning as the mapping table itself); the renewal-count case needed *no change at all* — 1243 held exactly, since that field was never touched.

The third case is genuinely CI-incompatible — confirmed by two separate live CI runs, for a different reason than first assumed. The "previously saved result" case depends on `eval_domain_avg_duration` surviving from round 1's own save-case, but round 1's `cleanup_table` step drops that table immediately after its case runs (needed so round 1 stays independently rerunnable) — so round 2 never has anything to find, even within the same job. A local "pass" earlier in this session was misleading: `pg_local`'s `agent_scratch` schema still held a real, months-old leftover table from actual Build 2.5 testing, untouched by this session's schema reload, coincidentally satisfying the case for reasons unrelated to anything built this session.

Fixed properly: an explicit `ci_skip` flag on that one case, checked against the `CI=true` env var GitHub Actions sets automatically — prints `[SKIPPED - CI]` rather than failing unpredictably or passing on stale coincidence.

Sample - real `eval_cases_round2.py` results after all fixes, local `CI=true` test run

`[SKIPPED - CI]` Do you have a previously saved result about average allocation duration by email domain? If so, which domain had the longest average duration?

`[PASS]` Which named patron department has the most recorded allocations?

answer: The named patron department with the most recorded allocations is "DRAW" with 128 allocations.

[PASS] How many allocations were never renewed (a renewal_count of zero)?

answer: There are 1,243 allocations that were never renewed.

=====
4/4 passed
=====

One more real exposure found and fixed in the same pass, unrelated to this session's own work:

PROJECT_DIAGRAMS/BUILD_3_FLOW/build3-handoff-brief.html/.pdf had contained a real department code as a hypothetical example question since Build 3 — already public long before this build. Found via the same -s pickaxe search used to verify the main purge. Fixed (swapped for its synthetic equivalent, per DEPARTMENT_MAP — not reproduced here, same reasoning as the mapping table itself; PDF regenerated and verified via pdftotext) and purged from history in the same pass.

Two different purge techniques used deliberately, not interchangeably. The old ci_fixture/ALLOCATION-fixture.csv was removed with --path ... --invert-paths (whole-file removal — it was superseded entirely). The Build 3 brief's one leaked string was fixed with --replace-text (in-place scrub, same technique as the original credential purge) — chosen specifically because that particular real code is unusual enough not to collide with other words, unlike some of the other real department codes, which are short enough to be substrings of common English/code words (e.g. one is a substring of "transform"/"transaction", which appear constantly throughout this codebase). That collision risk was checked directly via a full-history search before running anything, not assumed.

Full sequence of git-history rewrites this build, for the record: (1) the original leaked-credentials purge — --replace-text for the old hardcoded dev passwords (see environment-handoff-brief-v2.html's own "Open Item" section; not reproduced here on purpose), pushed to an empty remote so no force was needed on that push; (2) ci_fixture/ALLOCATION-fixture.csv removal, --path --invert-paths; (3) the Build 3 brief scrub, --replace-text. (2) and (3) both required git push --force, confirmed explicitly with the user before each one. Three separate backup bundles were taken along the way, all saved outside the repo entirely.

GitHub Setup, Step by Step

The actual sequence, including the parts that didn't go smoothly on the first try — kept in full because the troubleshooting is itself useful reference.

1. Handed over `gh repo create ... --push` as the intended one-liner — `gh` wasn't installed.
2. Established a hard boundary: creating a repo and authenticating to GitHub both require the user's own account — no browser access, no way to complete an OAuth/device-code flow in this session. What could be done: check for and install `gh` via `winget`. What only the user could do: `gh auth login`, and (as it turned out) the actual repo creation via the web UI.
3. `winget install --id GitHub.cli -e --accept-package-agreements --accept-source-agreements` → installed `gh 2.96.0`. `PATH` required a manual refresh (`$env:Path = [System.Environment]::GetEnvironmentVariable(...)`) in the already-open session to pick it up.
4. Repo created via the web UI by the user directly; `gh auth login` run by the user (their own credentials, own browser). Verified from this session via `gh auth status` → logged in, HTTPS protocol, correct token scopes.
5. `git remote -v / git fetch origin / git diff main origin/main --stat` confirmed the first push had already happened (by the user, outside this session's visibility) before this session ever ran `git push` itself.
6. `gh run list` → `gh run view <id>` → `gh run view <id> --log-failed` — the actual diagnostic sequence used every time a run needed investigating, rather than guessing from the YAML.
7. `gh run watch <id>` — streamed live step-by-step progress to confirm each fix before moving on, for every push this build.

Protocol choice: HTTPS, not SSH. `gh`'s credential helper handles auth automatically once logged in — no key generation/management needed for a single-machine, solo-author setup. SSH's advantage (multiple machines, a passphrase-separated key) doesn't apply here.

Verification Log

What was actually run against a real environment, not just read for plausibility — same standard as every build in this project, extended here to a genuinely fresh environment for the first time.

Local dry run, disposable throwaway container

Full migrate → ingest → transform → embed → pytest pipeline run against a fresh Postgres container (not `pg_local`, to avoid destroying real dev data) — this is what surfaced bugs 1 and 2 before they ever reached GitHub Actions.

First real Actions run

Failed on `pytest` only — every prior step (migrations, fixture ingest, transform, embed, doc-chunks) passed exactly as the local dry run predicted. Surfaced bug 3.

Second Actions run, post-fix

Fully green, every step including `pytest`.

Gate live-tested both directions

A relevant commit correctly triggered `eval`; an unrelated commit correctly skipped it.

eval job's first real run

Round 1 7/7; round 2 2/5 against the (still-being-replaced) old fixture — this run is what surfaced the anonymization gap's downstream eval consequences.

Full local reload onto the new official dataset

`raw/clean` rebuilt from `004/005`, re-ingested, `008/009` re-applied (rebuilding `clean.allocations` from scratch also drops the `summary_embedding` column those add — a real subtlety), re-embedded (1535 rows). `pytest` 19/19, `eval` round 1 7/7, `eval` round 2 5/5 (before the `ci_skip` fix existed).

Post-force-push Actions run

`test` ✓, `detect-changes` ✓, `eval` round 1 ✓ / round 2 4/5 (the one genuinely CI-incompatible case, confirmed live for the second time, not guessed).

Final Actions run, after the `ci_skip` fix

Fully green — `test`, `detect-changes`, `eval` (both rounds, the one case correctly skipped, not silently passing or failing).

History-purge verification, three times

Each `git filter-repo` pass followed by a direct full-history search (`git log --all -p | grep`, plus `-S` pickaxe and direct blob checks via `git show <commit>:<path>`) confirming zero remaining matches — never assumed clean from the tool's own success message alone.

✓ main CI · 30124608945

JOBS

✓ detect-changes in 5s

✓ test in 2m56s

✓ eval in 4m4s

✓ Run eval harness (round 1)

✓ Run eval harness (round 2)

Known Limitations & Open Items

question_id's concurrency-safety, flagged in Build 4, is still unexercised. CI's jobs run sequentially within a workflow (though in parallel with each other where needs: allows); nothing in this build actually ran two `ask_agent()` calls at the same literal moment. That real test still only becomes possible once Build 6 (a multi-user front end) exists.

Both eval rounds duplicate their full setup pipeline. Each GitHub Actions job gets its own fresh runner and service container, so the eval job can't reuse the test job's already-populated database — real duplicated compute (migrate/ingest/transform/embed runs twice on any commit that triggers eval). Acceptable at this project's scale since eval only fires on a relevant merge to main, not every push.

The living cheat sheet's `api_test.py` row is now stale (the file no longer exists) — addressed in the same session as this brief, see the cheat sheet's own changelog footer.

Environment State

COMPONENT	DETAIL
New Python	<code>synthesize_dataset.py</code>
Removed Python	<code>api_test.py</code> , <code>make_ci_fixture.py</code>
Changed Python	<code>ingest.py</code> (CSV_PATH default), <code>run_eval.py</code> (exit code, ci_skip), <code>eval_cases_round2.py</code> (DRAW keyword, ci_skip flag), <code>test_agent_scratch_boundary.py</code> , <code>test_data_quality.py</code>
New SQL	<code>008_add_summary_embedding.sql</code> – added CREATE EXTENSION IF NOT EXISTS vector
New data	<code>data/ALLOCATION-synthesized.csv</code> (1535 rows, official dataset) – replaces <code>ci_fixture/</code> (removed) and the real external CSV as the default
New local-only files	<code>department_mapping.local.py</code> , <code>DEPARTMENT_MAPPING.local.md</code> (both git-ignored – real department-code key, never committed)
New CI	<code>.github/workflows/ci.yml</code> – detect-changes, test, eval jobs; <code>pytest.ini</code>
New dependency (CI only)	<code>dorny/paths-filter@7b450fff21473bca461d4b92ce414b9d0420d706</code> (v4.0.2, pinned by SHA)
GitHub remote	<code>https://github.com/AndersonBallantyne/SQL_TEST_01</code> – public, HTTPS protocol
GitHub Actions secret	<code>ANTHROPIC_API_KEY</code> (set by the user directly, never handled in-session)
Diagram folder	<code>PROJECT_DIAGRAMS/BUILD_5_FLOW/</code> – phase1/phase3 flow diagrams, this brief
Backup bundles (outside the repo)	<code>C:\IMAGEBANK_2023\DOCKER_SQL\SQL_TEST_01_backups\</code> – 3 bundles, one per filter-repo pass

COMPONENT	DETAIL
Eval suite total	12 cases across 2 rounds; 11 run in CI (1 explicitly ci_skip'd), all passing
Project test suite	19/19 passing, both locally and in CI

Build 5 complete — all 3 phases, CI green end-to-end, public repo live with clean history. Next: the optional Build 6 (Streamlit front end). Ongoing cross-build command reference lives in `sql-test-01-cheatsheet.html`.